

MemLab 16 MiB

Manual técnico

Simulador interactivo de gestión de memoria principal

Versión documentada: estado local auditado el 23 de septiembre de 2026
Commit base observado: 20cec7a
Plataforma: HTML5, CSS y JavaScript estándar
Espacio simulado: 16 MiB (2^{24} bytes)

Contenido

1	Propósito, alcance y trazabilidad	3
1.1	Inventario auditado	3
2	Arquitectura del sistema	3
2.1	Estilo arquitectónico	3
2.2	Flujo de control	4
2.3	Responsabilidades por capa	4
3	Modelo de dominio y estructuras de datos	4
3.1	Unidades y espacio de direcciones	4
3.2	Configuración <code>cfg</code>	4
3.3	Bloque de memoria	5
3.4	Proceso simulado	5
4	Implementación explicada: código esencial	5
4.1	Estado global, unidades y segmentos de un proceso	6
4.2	Cómo se particiona la memoria	6
4.3	Selección: primer, mejor y peor ajuste	7
4.4	Carga y división de un hueco dinámico	8
4.5	Liberación y fusión de huecos contiguos	9
4.6	Compactación de memoria dinámica	10
4.7	Cálculo de uso y fragmentación	10
4.8	Cómo se distribuyen código, datos, pila y heap	11
4.9	Actualización coherente de la interfaz	11
5	Algoritmos de gestión de memoria	12
5.1	Particiones estáticas de tamaño fijo	12
5.2	Particiones estáticas de tamaño variable	12
5.3	Particiones dinámicas	12
5.4	Primer, mejor y peor ajuste	12
5.5	Fusión de huecos	13
5.6	Compactación	13
5.7	Métricas	13
6	Catálogo completo de funciones	13
7	Renderizado e interfaz	15
7.1	Mapa proporcional	15
7.2	Detalle de segmentos	15

7.3 Bitácora	15
8 Eventos y ciclo de vida	15
9 Ejecución, despliegue y compatibilidad	16
9.1 Ejecución directa	16
9.2 Requisitos	16
9.3 Despliegue estático	16
10 Validación realizada	16
11 Seguridad, accesibilidad y calidad	17
11.1 Controles existentes	17
11.2 Hallazgos y limitaciones	17
12 Mantenimiento y extensión	17
12.1 Puntos de cambio habituales	18
12.2 Pruebas recomendadas	18
13 Conclusión técnica	18

1 Propósito, alcance y trazabilidad

Resumen técnico

MemLab es una aplicación web educativa, estática y de una sola página que modela la asignación contigua de memoria sobre 16 MiB. Implementa particiones estáticas iguales, particiones estáticas de tamaños variables y particiones dinámicas; permite comparar primer, mejor y peor ajuste, observar fragmentación y ejecutar compactación. No contiene servidor, base de datos, autenticación ni persistencia.

Este manual explica la arquitectura, el modelo de datos, los algoritmos, las funciones, el renderizado, la ejecución, las dependencias y las restricciones del estado actual del proyecto. La revisión se hizo sobre la copia de trabajo visible, incluidas sus modificaciones locales, sin alterar los cuatro archivos originales.

1.1 Inventario auditado

Archivo	Líneas	Responsabilidad
README.md	105	Visión general, ejecución y explicación conceptual.
index.html	224	Estructura semántica, controles, paneles, tabla, mapa y carga de recursos.
css/style.css	331	Sistema visual, temas claro/oscuro, estados de bloques, tablas y adaptación responsiva.
js/script.js	783	Estado, algoritmos de memoria, métricas, renderizado, bitácora y eventos.

El código ejecutable suma 1 338 líneas entre HTML, CSS y JavaScript. El archivo JavaScript declara 33 funciones con nombre, más una función auxiliar de selección del DOM y diversas funciones anónimas usadas como callbacks.

2 Arquitectura del sistema

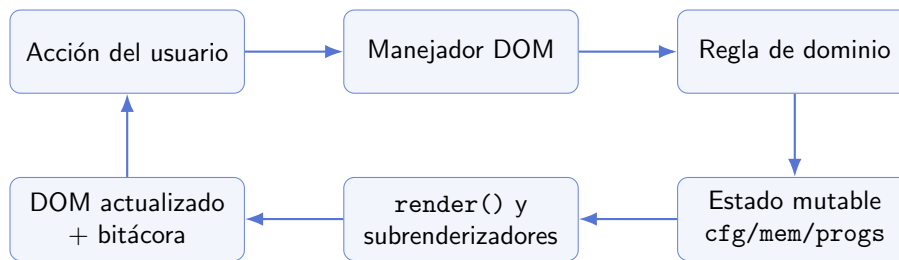
2.1 Estilo arquitectónico

La solución sigue una arquitectura cliente monolítica y dirigida por eventos, encapsulada en una IIFE (*Immediately Invoked Function Expression*). Puede entenderse como una separación ligera entre modelo, controladores y vistas:

- **Modelo en memoria:** `cfg`, `mem`, `progs`, `bitacora` y contadores auxiliares.
- **Lógica de dominio:** construcción de memoria, selección de huecos, carga, liberación, fusión, compactación y cálculo de métricas.
- **Controladores:** escuchadores DOM que traducen clics y cambios de formulario en mutaciones del modelo.
- **Vista:** funciones `render*` que regeneran HTML a partir del estado actual.

No existe un framework, enrutador, API ni almacén externo. El navegador contiene todo el ciclo de vida y un refresco de página reinicia el sistema.

2.2 Flujo de control



El patrón dominante es: evento $\rightarrow$ mutación sincrónica $\rightarrow$ `render()`. El renderizado completo es razonable porque el estado es pequeño: decenas de bloques y procesos, no miles.

2.3 Responsabilidades por capa

HTML. Define la estructura estable y los puntos de montaje identificados por 33 IDs únicos. Incluye atributos ARIA en grupos, controles y bitácora.

CSS. Usa variables personalizadas como tokens de diseño, sigue el esquema de color del sistema, reduce el mapa a una columna en pantallas de hasta 900 px y respeta la preferencia de movimiento reducido.

JavaScript. Mantiene el estado privado dentro de la IIFE, procesa reglas de memoria, genera fragmentos HTML y conecta 14 grupos de interacciones.

3 Modelo de dominio y estructuras de datos

3.1 Unidades y espacio de direcciones

Las constantes principales son:

$$\text{KiB} = 1024, \quad \text{MiB} = 1\,048\,576, \quad \text{TOTAL} = 16 \times \text{MiB} = 16\,777\,216 \text{ bytes.}$$

Las direcciones válidas son 0x000000 a 0xFFFFF. Todos los rangos mostrados son inclusivos, por lo que el extremo superior de un bloque es $base + size - 1$. La memoria de usuario es:

$$M_u = \text{TOTAL} - \text{cfg.so.}$$

3.2 Configuración `cfg`

Campo	Valor inicial	Significado
<code>metodo</code>	"fija"	Estrategia activa: fija, variable o dinámica.
<code>algo</code>	"primero"	Política de selección: primer, mejor o peor ajuste.
<code>so</code>	2 MiB	Memoria residente reservada al sistema operativo.
<code>nFijas</code>	7	Cantidad de particiones iguales.
<code>variables</code>	512, 1024, 1536, 2048, 3072, 6144 KiB	Tabla inicial de particiones distintas; suma 14 MiB.

Campo	Valor inicial	Significado
<code>autoCompact</code>	<code>false</code>	Compacta al fallar por falta de continuidad si el total libre alcanza.

3.3 Bloque de memoria

Cada elemento de `mem` es un objeto con `id`, `base`, `size`, `tipo`, `pid` y `fija`; las particiones estáticas añaden `idx`. Los tipos posibles son:

- `so`: bloque reservado al sistema operativo;
- `particion`: límite estático, libre u ocupado según `pid`;
- `hueco`: espacio libre dinámico;
- `proceso`: bloque dinámico ocupado del tamaño exacto;
- `reservado`: sobrante no particionado e inutilizable.

El invariante esperado es que los bloques estén ordenados por dirección, no se solapen y cubran todo el rango físico; los límites de las particiones estáticas no se desplazan. En dinámica, `fusionar()` mantiene la propiedad de que no queden dos huecos contiguos separados después de una liberación.

3.4 Proceso simulado

Un proceso contiene `pid`, etiqueta `Pn`, nombre, segmentos, tamaño total en bytes, color, estado, base y bloque. Su tamaño es:

$$size = (codigo + datos + pila + heap) \times 1024.$$

Los estados son `cola`, `cargado` y `rechazado`. Un rechazo no elimina el trabajo: una liberación o compactación llama a `limpiarRechazos()` y lo devuelve a la cola.

Programa	Código	Datos	Pila	Heap	Total KiB
Editor de texto	640	256	64	64	1 024
Compilador C	1 280	512	128	128	2 048
Navegador web	1 408	1 024	256	1 152	3 840
Motor de base de datos	768	1 536	128	640	3 072
Terminal	192	96	32	128	448
Render 3D	1 024	2 048	256	2 816	6 144
Servicio de red	224	128	32	384	768
Hoja de cálculo	512	768	64	192	1 536

El catálogo suma 18 880 KiB, deliberadamente más que los 14 MiB de usuario de la configuración predeterminada; esto permite observar admisiones y rechazos.

4 Implementación explicada: código esencial

Esta sección reproduce y explica el núcleo real de `js/script.js`. Se incluye el código que determina el comportamiento del simulador; se omiten principalmente cadenas de presentación, estilos y marcado repetitivo. La numeración de cada listado es local al fragmento, mientras que el nombre de la función permite encontrar su versión completa en el archivo fuente.

4.1 Estado global, unidades y segmentos de un proceso

```

1 var KiB = 1024, MiB = 1048576, TOTAL = 16 * MiB;
2
3 var SEGS = [
4   { k:"codigo", n:"Codigo",   c:"s-cod" },
5   { k:"datos",  n:"Datos",    c:"s-dat" },
6   { k:"pila",   n:"Pila",     c:"s-pil" },
7   { k:"heap",   n:"Monticulo", c:"s-hea" }
8 ];
9
10 var cfg = {
11   metodo:"fija", algo:"primero", so:2 * MiB, nFijas:7,
12   variables:[512, 1024, 1536, 2048, 3072, 6144],
13   autoCompact:false
14 };
15
16 var mem = [], progs = [], bitacora = [];
17 var reloj = 0, sel = null, uid = 1, pid = 1;

```

Listing 1: Unidades, orden de segmentos y configuración inicial.

Lectura del código. TOTAL fija un espacio de 2^{24} bytes. SEGS no guarda los valores de un proceso: define el orden canónico con el que se suman y se dibujan código, datos, pila y montículo. cfg reúne las decisiones que alteran la topología de memoria. mem contiene los bloques físicos simulados y progs contiene los procesos; se mantienen separados y se relacionan mediante pid.

```

1 function nuevoPrograma(nombre, segs){
2   var total = 0;
3   for (var i=0; i<SEGS.length; i++){
4     total += segs[SEGS[i].k];
5   }
6   return {
7     pid: pid,
8     etq: "P" + (pid++),
9     nombre: nombre,
10    segs: segs,
11    size: total * KiB,
12    color: COLORES[(pid - 2) % COLORES.length],
13    estado: "cola",
14    base: null,
15    blk: null
16  };
17 }

```

Listing 2: Creación de un proceso y cálculo de su tamaño.

Los tamaños introducidos por la interfaz están en KiB. El bucle suma los cuatro segmentos y luego multiplica una sola vez por 1 024 para guardar size en bytes. base y blk permanecen nulos hasta la admisión. Esta separación evita mezclar unidades y permite calcular direcciones exactas.

4.2 Cómo se particiona la memoria

```

1 function tamFija(){
2   var usuario = TOTAL - cfg.so;
3   return Math.floor(
4     Math.floor(usuario / cfg.nFijas) / (4 * KiB)
5   ) * (4 * KiB);
6 }

```

Listing 3: Tamaño de una partición fija igual.

Primero se descuenta el S.O.; después se divide la memoria de usuario entre (N). El segundo redondeo fuerza que la partición sea múltiplo de 4 KiB. Si la división no es exacta, los bytes restantes no desaparecen: `construirMemoria()` los registra como zona reservada.

```

1 function construirMemoria(motivo){
2   mem = [];
3   mem.push({ id:uid++, base:0, size:cfg.so,
4             tipo:"so", pid:null, fija:true });
5   var base = cfg.so, i;
6
7   if (cfg.metodo === "fija"){
8     var ps = tamFija();
9     for (i=0; i<cfg.nFijas; i++){
10      mem.push({ id:uid++, base:base, size:ps,
11               tipo:"particion", idx:i+1, pid:null, fija:true });
12      base += ps;
13    }
14  } else if (cfg.metodo === "variable"){
15    for (i=0; i<cfg.variables.length; i++){
16      var s = cfg.variables[i] * KiB;
17      if (base + s > TOTAL) break;
18      mem.push({ id:uid++, base:base, size:s,
19               tipo:"particion", idx:i+1, pid:null, fija:true });
20      base += s;
21    }
22  } else {
23    mem.push({ id:uid++, base:base, size:TOTAL-base,
24             tipo:"hueco", pid:null, fija:false });
25    base = TOTAL;
26  }
27
28  if (base < TOTAL){
29    mem.push({ id:uid++, base:base, size:TOTAL-base,
30             tipo:"reservado", pid:null, fija:true });
31  }
32  for (i=0; i<progs.length; i++){
33    progs[i].estado = "cola";
34    progs[i].base = null;
35    progs[i].blk = null;
36  }
37  sel = null;
38  if (motivo) addLog("config", motivo);
39 }

```

Listing 4: Construcción de los tres esquemas de memoria.

El puntero local `base` actúa como la siguiente dirección libre. En fija, avanza siempre `ps`; en variable, avanza el tamaño indicado por la tabla; en dinámica, toda la memoria de usuario empieza como un único hueco. El chequeo `base + s > TOTAL` impide crear una partición fuera del rango físico. Una reconfiguración invalida las direcciones anteriores, por eso todos los programas regresan a cola.

4.3 Selección: primer, mejor y peor ajuste

```

1 function libres(){
2   return mem.filter(function(b){
3     return b.pid === null &&
4            (b.tipo === "particion" || b.tipo === "hueco");
5   });
6 }
7
8 function elegir(cands){
9   if (cfg.metodo === "fija") return cands[0];
10  if (cfg.algo === "primero") return cands[0];

```

```

11  var best = cands[0];
12  for (var i=1; i<cands.length; i++){
13      if (cfg.algo === "mejor"
14          ? cands[i].size < best.size
15          : cands[i].size > best.size){
16          best = cands[i];
17      }
18  }
19  return best;
20  }

```

Listing 5: Obtención y selección de bloques candidatos.

`libres()` excluye explícitamente el S.O. y la zona reservada. Antes de llamar a `elegir()`, `cargar()` filtra además por capacidad. Por eso `cands[0]` siempre es un bloque válido. Mejor ajuste conserva el menor; peor ajuste conserva el mayor. En fija todos miden igual y se usa el primero.

4.4 Carga y división de un hueco dinámico

```

1  function cargar(id, silencioso){
2      var p = prog(id);
3      if (!p || p.estado === "cargado") return false;
4      var l = libres();
5      var cands = l.filter(function(b){ return b.size >= p.size; });
6
7      if (!cands.length && cfg.metodo === "dinamica" &&
8          cfg.autoCompact){
9          var totalLibre = l.reduce(function(a,b){
10              return a + b.size;
11          }, 0);
12          if (totalLibre >= p.size){
13              compactar(true);
14              l = libres();
15              cands = l.filter(function(b){ return b.size >= p.size; });
16          }
17      }
18
19      if (!cands.length){
20          p.estado = "rechazado";
21          return false;
22      }
23
24      var b = elegir(cands);
25      if (b.fija){
26          b.pid = p.pid;
27      } else {
28          if (b.size > p.size){
29              var resto = {
30                  id:uid++, base:b.base + p.size,
31                  size:b.size - p.size,
32                  tipo:"hueco", pid:null, fija:false
33              };
34              b.size = p.size;
35              mem.splice(mem.indexOf(b) + 1, 0, resto);
36          }
37          b.tipo = "proceso";
38          b.pid = p.pid;
39      }
40      p.estado = "cargado";
41      p.base = b.base;
42      p.blk = b.id;
43      return true;
44  }

```

Listing 6: Búsqueda, compactación automática y admisión.

La condición de admisión es `b.size >= p.size`. Si ningún hueco continuo alcanza, la compactación automática solo se justifica cuando la suma libre sí es suficiente. En una partición estática se asigna el PID sin cambiar el límite; la diferencia queda como fragmentación interna. En dinámica se crea `resto` con base `b.base + p.size`, se reduce el bloque original al tamaño exacto del proceso y se inserta el hueco justo después. Así se conserva el orden de direcciones y la fragmentación interna es cero.

Ejemplo numérico de división

Si un hueco empieza en `0x200000`, mide 4 MiB y recibe un proceso de 1 536 KiB, el proceso conserva la base `0x200000`. El nuevo hueco comienza en `0x380000` y mide 2 560 KiB. Los dos tamaños suman exactamente los 4 MiB originales; no se crean ni se pierden bytes.

4.5 Liberación y fusión de huecos contiguos

```

1 function liberar(id, silencioso){
2   var p = prog(id);
3   if (!p || p.estado !== "cargado") return;
4   var b = null;
5   for (var i=0; i<mem.length; i++){
6     if (mem[i].pid === p.pid) b = mem[i];
7   }
8   if (!b) return;
9
10  b.pid = null;
11  var fusiones = 0;
12  if (!b.fija){
13    b.tipo = "hueco";
14    fusiones = fusionar();
15  }
16  p.estado = "cola";
17  p.base = null;
18  p.blk = null;
19  limpiarRechazos();
20 }
```

Listing 7: Liberación de un proceso.

La liberación rompe el vínculo en ambos sentidos: el bloque pierde su PID y el proceso pierde base e identificador de bloque. Una partición estática conserva su tamaño. Un bloque dinámico recupera el tipo hueco y activa la fusión.

```

1 function fusionar(){
2   var fusiones = 0;
3   for (var i=0; i<mem.length-1; ){
4     if (mem[i].tipo === "hueco" &&
5         mem[i+1].tipo === "hueco"){
6       mem[i].size += mem[i+1].size;
7       mem.splice(i+1, 1);
8       fusiones++;
9     } else {
10      i++;
11    }
12  }
13  return fusiones;
14 }
```

Listing 8: Coalescencia de huecos vecinos.

El índice no avanza después de fusionar. Esto es intencional: si había tres huecos seguidos, el bloque recién ampliado vuelve a compararse con el siguiente y absorbe toda la cadena. La base del primero se conserva y el segundo se elimina.

4.6 Compactación de memoria dinámica

```

1 function compactar(auto){
2   if (cfg.metodo !== "dinamica") return;
3   var ocupados = mem.filter(function(b){
4     return b.tipo === "proceso" && b.pid !== null;
5   }).sort(function(a,b){ return a.base - b.base; });
6   if (!ocupados.length) return;
7
8   var base = cfg.so, movidos = 0;
9   for (var i=0; i<ocupados.length; i++){
10    if (ocupados[i].base !== base) movidos++;
11    ocupados[i].base = base;
12    var p = prog(ocupados[i].pid);
13    if (p) p.base = base;
14    base += ocupados[i].size;
15  }
16
17  var so = mem.filter(function(b){ return b.tipo === "so"; });
18  mem = so.concat(ocupados);
19  if (base < TOTAL){
20    mem.push({ id:uid++, base:base, size:TOTAL-base,
21              tipo:"hueco", pid:null, fija:false });
22  }
23  limpiarRechazos();
24 }

```

Listing 9: Reubicación de procesos y creación de un hueco final.

Ordenar por base conserva el orden relativo de los procesos. El acumulador **base** comienza exactamente al final del S.O. y aumenta con cada tamaño, por lo que no quedan huecos intermedios. Es indispensable actualizar tanto el bloque como **p.base**; de otro modo, el mapa y el registro base mostrarían direcciones distintas. Al final se reconstruye **mem**: S.O., procesos contiguos y como máximo un hueco.

4.7 Cálculo de uso y fragmentación

```

1 function metricas(){
2   var usuario = TOTAL - cfg.so;
3   var usado = 0, fragInt = 0, i, p;
4   for (i=0; i<mem.length; i++){
5     if (mem[i].pid !== null){
6       p = prog(mem[i].pid);
7       if (p){
8         usado += p.size;
9         fragInt += mem[i].size - p.size;
10      }
11    }
12  }
13  var l = libres();
14  var libreTotal = l.reduce(function(a,b){
15    return a + b.size;
16  }, 0);
17  var mayor = l.length ? Math.max.apply(null,
18    l.map(function(b){ return b.size; })) : 0;
19  var fragExt = cfg.metodo === "dinamica"
20    ? libreTotal - mayor : libreTotal;

```

```

21  return {
22      usuario:usuario, usado:usado, fragInt:fragInt,
23      libreTotal:libreTotal, mayor:mayor,
24      huecos:l.length, fragExt:fragExt,
25      util:usuario ? usado / usuario : 0
26  };
27  }

```

Listing 10: Métricas derivadas del estado de memoria.

usado suma tamaños reales de procesos, no tamaños de partición. **fragInt** suma el sobrante dentro de cada bloque ocupado. En dinámica esa diferencia siempre es cero gracias a la división exacta. **fragExt** resta el mayor hueco al total libre: representa los bytes libres que no ayudan a cargar un proceso limitado por el mayor tramo continuo.

4.8 Cómo se distribuyen código, datos, pila y heap

```

1  var off = 0;
2  for (i=0; i<SEGS.length; i++){
3      var s = SEGS[i];
4      var sz = p.segs[s.k] * KiB;
5      var inicio = p.base + off;
6      var fin = p.base + off + sz - 1;
7      // La interfaz muestra: s.n, sz, off, inicio y fin.
8      off += sz;
9  }

```

Listing 11: Cálculo de desplazamientos y rangos de cada segmento.

El simulador usa un modelo pedagógico contiguo: dentro del bloque del proceso se ubican, en ese orden, código, datos, pila y heap. **off** comienza en cero; el inicio de cada segmento es **p.base + off** y el final es inclusivo. Al sumar **sz**, el siguiente segmento comienza exactamente un byte después.

Qué significa “particionar la pila” en este proyecto

La pila no se administra como una partición física independiente ni crece durante la ejecución. Es un segmento de tamaño fijo, expresado en KiB, dentro del bloque contiguo asignado al proceso. El programa visualiza su rango y proporción, pero no simula marcos de activación, punteros de pila, desbordamiento ni crecimiento hacia direcciones menores. Esta precisión evita atribuir al simulador un comportamiento que el código no implementa.

4.9 Actualización coherente de la interfaz

```

1  function render(){
2      renderControles();
3      renderKPIs();
4      renderMapa();
5      renderDetalle();
6      renderProgs();
7      renderLog();
8  }

```

Listing 12: Punto central de renderizado.

Después de cada mutación importante, **render()** vuelve a derivar controles, indicadores, mapa, detalle, tabla y bitácora del mismo estado. No hay copias independientes de los datos en la vista; esta decisión reduce inconsistencias en una aplicación pequeña y sin framework.

5 Algoritmos de gestión de memoria

5.1 Particiones estáticas de tamaño fijo

La memoria de usuario se divide en N partes iguales, redondeadas hacia abajo a múltiplos de 4 KiB:

$$P = \left\lfloor \frac{\lfloor M_u/N \rfloor}{4 \text{ KiB}} \right\rfloor \times 4 \text{ KiB}.$$

Se crea una partición por cada índice; cualquier resto queda como zona **reservado**. La carga usa la primera partición libre con capacidad suficiente. El selector de ajuste se deshabilita porque todas las particiones tienen el mismo tamaño. La fragmentación interna de un proceso es $P - \text{size}$.

Con 2 MiB de S.O. y 7 particiones, $P = 2$ MiB y no queda sobrante. En el arranque se cargan cuatro procesos que ocupan 5 056 KiB útiles dentro de 8 192 KiB asignados; la fragmentación interna inicial es 3 136 KiB.

5.2 Particiones estáticas de tamaño variable

`construirMemoria()` recorre `cfg.variables` y crea una partición por tamaño, siempre que su extremo no exceda `TOTAL`; al primer tamaño que no cabe se interrumpe el bucle. Los tamaños añadidos por la interfaz se ordenan de menor a mayor. Un proceso ocupa una partición completa y produce fragmentación interna igual a tamaño de partición – *size*.

Aquí sí se usa la política configurada. Las particiones libres no pueden unirse ni compactarse porque sus límites son parte de la configuración estática.

5.3 Particiones dinámicas

El modelo comienza con un único hueco de M_u . Al asignar un proceso:

1. se filtran los huecos cuyo tamaño es al menos el del proceso;
2. se selecciona uno según la política activa;
3. si el hueco es mayor, se divide en un bloque de proceso exacto y un hueco residual situado inmediatamente después;
4. el bloque elegido cambia de **hueco** a **proceso**.

Por construcción no existe fragmentación interna. La liberación convierte el bloque en hueco y fusiona vecinos libres contiguos. La fragmentación externa aparece cuando el total libre se reparte entre varios huecos.

5.4 Primer, mejor y peor ajuste

Sea C la lista, en orden de dirección, de bloques libres que pueden contener al proceso:

Primer ajuste. Devuelve C_0 . Costo de búsqueda completo: $O(B)$, donde B es la cantidad de bloques; la selección posterior es $O(1)$.

Mejor ajuste. Recorre C y conserva el candidato de menor tamaño. Costo $O(B)$; busca minimizar el resto inmediato.

Peor ajuste. Recorre C y conserva el candidato de mayor tamaño. Costo $O(B)$; busca dejar un resto grande potencialmente reutilizable.

No se mantienen índices, árboles ni listas segregadas; el objetivo es claridad didáctica, no escalabilidad de un gestor real.

5.5 Fusión de huecos

`fusionar()` recorre `mem` de izquierda a derecha. Si dos posiciones consecutivas son huecos, suma sus tamaños y elimina la segunda sin avanzar el índice; así puede absorber cadenas de tres o más huecos. Su costo es conceptualmente $O(B)$, aunque cada `splice` desplaza elementos y da un peor caso $O(B^2)$ en una implementación de arreglo. Para la escala de la simulación es irrelevante.

5.6 Compactación

La compactación solo actúa en modo dinámico:

1. obtiene los bloques de proceso y los ordena por base;
2. coloca el primero justo después del S.O. y los demás de forma contigua;
3. actualiza la base tanto del bloque como del proceso;
4. reconstruye `mem` como S.O. + procesos + un hueco final, si sobra;
5. devuelve procesos rechazados a cola y registra el número de movimientos.

La complejidad es $O(P \log P + B)$ por la ordenación de P procesos y los filtrados sobre B bloques. La compactación automática se dispara únicamente si no existe hueco suficiente pero la suma libre sí alcanza el tamaño solicitado.

5.7 Métricas

`metricas()` calcula:

$$\begin{aligned} usado &= \sum size(proceso\ cargado), \\ fragInt &= \sum (size(bloque) - size(proceso)), \\ libreTotal &= \sum size(bloque\ libre), \\ mayor &= \max size(bloque\ libre), \\ utilizacion &= usado/M_u. \end{aligned}$$

En modo dinámico, $fragExt = libreTotal - mayor$, que expresa cuánto espacio libre no pertenece al mayor tramo contiguo. En modos estáticos, el código presenta todo el espacio de particiones libres como fragmentación externa, bajo la interpretación didáctica de que esas particiones no pueden combinarse.

6 Catálogo completo de funciones

Las complejidades usan B =bloques, P =programas y $S = 4$ =segmentos.

Función	Contrato y efectos	Costo
<code>hex()</code>	Convierte un número a dirección hexadecimal mayúscula de seis dígitos. No muta estado.	$O(1)$
<code>miles()</code>	Inserta separadores finos de millares en una representación decimal.	$O(d)$
<code>fmt()</code>	Formatea bytes como KiB o MiB, con hasta dos decimales en MiB.	$O(d)$
<code>prog()</code>	Busca por PID en <code>progs</code> ; devuelve objeto o <code>null</code> .	$O(P)$

Función	Contrato y efectos	Costo
<code>esc()</code>	Escapa <code>&</code> , <code><</code> , <code>></code> y comillas para inserción HTML.	$O(n)$
<code>addLog()</code>	Incrementa el reloj, antepone un evento y limita la bitácora a 80 filas.	$O(P)$ por <code>unshift</code>
<code>nuevoPrograma()</code>	Suma cuatro segmentos, asigna PID, etiqueta, color y estado inicial. Incrementa <code>pid</code> .	$O(S)$
<code>sembrarProgramas()</code>	Reinicia PID y recrea los ocho programas base con copias de segmentos.	$O(P)$
<code>tamFija()</code>	Calcula el tamaño igual redondeado a 4 KiB.	$O(1)$
<code>construirMemoria()</code>	Reconstruye bloques según el método, devuelve procesos a cola, limpia selección y opcionalmente registra el motivo.	$O(B + P)$
<code>libres()</code>	Filtra particiones o huecos sin PID. Excluye S.O. y reservado.	$O(B)$
<code>elegir()</code>	Aplica primera, mínima o máxima capacidad sobre candidatos.	$O(B)$
<code>nombreBloque()</code>	Produce una descripción humana según el tipo.	$O(1)$
<code>cargar()</code>	Busca candidatos, compacta si corresponde, explica rechazos, divide huecos dinámicos y enlaza proceso/bloque.	$O(B + P)$, más compactación
<code>liberar()</code>	Localiza el bloque, limpia la asignación, fusiona huecos dinámicos y restablece rechazos.	$O(B + P)$
<code>fusionar()</code>	Une huecos adyacentes y devuelve el número de fusiones.	hasta $O(B^2)$
<code>limpiarRechazos()</code>	Convierte todos los estados rechazados en cola.	$O(P)$
<code>compactar()</code>	Reubica procesos dinámicos hacia direcciones bajas y crea un hueco final. Registra resultado.	$O(P \log P + B)$
<code>metricas()</code>	Agrega uso, fragmentación, huecos, reserva, procesos y utilización.	$O(BP)$ por búsquedas PID
<code>\$(selector)</code>	Alias privado de <code>document.querySelector</code> .	según DOM
<code>render()</code>	Orquesta controles, KPIs, mapa, detalle, programas y bitácora.	suma de subrenders
<code>renderControles()</code>	Sincroniza pestañas, algoritmo, panel visible, textos, botones y chips.	$O(V)$
<code>renderKPIs()</code>	Calcula métricas y monta cinco tarjetas.	$O(BP)$
<code>kpi()</code>	Devuelve el HTML de una tarjeta y barra opcional.	$O(1)$
<code>renderMapa()</code>	Genera un botón proporcional por bloque, fragmentación interna y clases por altura.	$O(BP)$
<code>renderDetalle()</code>	Muestra rango, registro base/límite y tabla de segmentos del bloque seleccionado.	$O(B + P + S)$
<code>segbar()</code>	Genera la barra proporcional de cuatro segmentos.	$O(S)$
<code>renderProgs()</code>	Regenera filas, acciones, estados y suma del catálogo.	$O(PS)$
<code>renderLog()</code>	Regenera hasta 80 eventos en orden inverso cronológico.	$O(L)$

Función	Contrato y efectos	Costo
<code>cargarTodos()</code>	Intenta cargar los trabajos no cargados en orden y registra el total admitido.	$O(P(B + P))$
<code>liberarTodos()</code>	Libera cada proceso cargado, registra cada efecto y el resumen.	$O(P(B + P))$
<code>escenario Fragmentacion()</code>	Reinicia, carga en orden, libera posiciones alternas y sugiere un proceso pendiente.	$O(P(B + P))$
<code>reiniciar()</code>	Restaura catálogo y memoria conservando la configuración activa.	$O(B + P)$
<code>arranque()</code>	Inicializa el sistema, carga cuatro programas, selecciona el primer bloque y renderiza.	$O(P(B + P))$

7 Renderizado e interfaz

7.1 Mapa proporcional

Cada bloque es un botón con **flex-grow** igual a su tamaño en bytes. Dentro de un bloque ocupado estático, el contenido útil y la franja de fragmentación interna reciben proporciones independientes. Después de insertar el DOM, `renderMapa()` mide la altura real y activa clases **tight** o **micro** para reducir texto en bloques pequeños.

El mapa no dibuja píxeles por byte: normaliza mediante Flexbox dentro de una altura CSS limitada por `clamp(430px, 70vh, 780px)`. Todos los valores numéricos y rangos sí provienen del modelo exacto.

7.2 Detalle de segmentos

Cuando se selecciona un bloque ocupado, el desplazamiento empieza en cero y se incrementa en el orden código, datos, pila y heap. Para cada segmento se muestra:

$$inicio = base + offset, \quad fin = inicio + tamaño - 1.$$

El llamado “registro límite” mostrado por la interfaz es el tamaño del proceso en hexadecimal, no la última dirección absoluta. Esto corresponde al uso didáctico de un par base/límite.

7.3 Bitácora

Los eventos se anteponen con marcas `t001`, `t002`, etc. y se clasifican como configuración, asignación, liberación, rechazo, compactación, consulta o nota. Solo se conservan los 80 más recientes. La bitácora es estado volátil y no se exporta.

8 Eventos y ciclo de vida

Control	Efecto
Pestaña de método	Cambia <code>cfg.metodo</code> , reconstruye toda la memoria y devuelve procesos a cola.

Control	Efecto
Selector de algoritmo	Cambia <code>cfg.algo</code> ; registra la justificación.
Tamaño de S.O.	Convierte MiB a bytes y reconstruye memoria.
Número de particiones	Actualiza <code>nFijas</code> y reconstruye.
Chips variables	Eliminan por índice y reconstruyen.
Añadir partición	Valida mínimo y capacidad restante, ordena y reconstruye.
Compactar / automático	Compacta inmediatamente o activa el reintento automático.
Cargar / liberar	Invoca la regla sobre el PID de la fila y renderiza.
Clic en mapa	Alterna selección y registra una consulta.
Nuevo programa	Normaliza segmentos, valida contra memoria de usuario y agrega a cola.
Escenario	Recrea catálogo, carga lo posible y libera procesos alternos.
Reiniciar	Recrea estado operativo, pero conserva método y opciones actuales.

9 Ejecución, despliegue y compatibilidad

9.1 Ejecución directa

No hay instalación ni compilación. Desde la raíz puede abrirse `index.html` en un navegador moderno. También puede servirse localmente:

```

1 cd /ruta/a/memlab
2 python3 -m http.server 8000
3 # abrir http://localhost:8000/

```

La ejecución sobre HTTP facilita depuración y evita restricciones particulares de `file://`. Las fuentes de Google son la única dependencia de red; si no están disponibles, la pila tipográfica usa fuentes del sistema y la funcionalidad permanece intacta.

9.2 Requisitos

- navegador con JavaScript habilitado, Flexbox, Grid, variables CSS y `color-mix`; se recomienda una versión moderna de Chrome, Edge, Firefox o Safari;
- resolución flexible; a 900 px o menos la cuadrícula pasa a una columna;
- no se requieren Node.js, paquetes npm, servidor de aplicación ni base de datos.

9.3 Despliegue estático

Puede publicarse tal cual en GitHub Pages, Netlify, Vercel estático, S3 u otro servidor de archivos. Debe conservarse la relación de rutas `index.html`, `css/style.css` y `js/script.js`. No existen variables de entorno ni secretos.

10 Validación realizada

Prueba	Resultado	Evidencia
Sintaxis JavaScript	Correcto	<code>node --check js/script.js</code> sin errores.
Servicio HTTP	Correcto	<code>index.html</code> , CSS y JS respondieron 200 con MIME apropiado.
Selectores DOM	Correcto	33 IDs únicos; todos los selectores <code>\$("#id")</code> existen.
Duplicados de ID	Correcto	No se detectaron IDs repetidos.
Revisión estática	Correcto	Se trazaron configuración, mutaciones, renderizado y eventos.
Prueba Playwright	No ejecutada	La dependencia Python Playwright no está instalada en el entorno.

11 Seguridad, accesibilidad y calidad

11.1 Controles existentes

`esc()` protege el nombre de usuario al representarlo en mapa, detalle y tabla. Los controles principales cuentan con etiquetas ARIA, el registro usa `aria-live="polite"`, hay foco visible y la animación solo se aplica cuando el usuario no solicita reducción de movimiento.

11.2 Hallazgos y limitaciones

1. **Inserción HTML en bitácora.** El nombre de un programa añadido se concatena sin escape al texto del evento y `renderLog()` lo inserta con `innerHTML`. En un contexto compartido, un nombre con marcado podría convertirse en XSS DOM. Debe escaparse al registrar o al renderizar.
2. **Sin persistencia.** Recargar la página elimina configuración, programas agregados, asignaciones y bitácora.
3. **Tabla variable frente al S.O.** Cambiar el S.O. puede dejar una tabla configurada cuya suma exceda la memoria de usuario. El constructor ignora desde la primera partición que no cabe, mientras el texto de suma conserva toda la lista; conviene normalizarla o marcar explícitamente entradas omitidas.
4. **Mensaje de rechazo fijo.** En ciertos casos con particiones libres demasiado pequeñas, el texto puede afirmar que “todas están ocupadas” cuando el problema real es capacidad individual insuficiente.
5. **Compactación con memoria llena.** Si no queda espacio, el registro puede indicar “1 hueco de 0 KiB” aunque el arreglo no contenga hueco final.
6. **Métrica didáctica.** En estáticas, llamar fragmentación externa a toda partición libre es una convención pedagógica; no es una definición universal.
7. **Sin suite automatizada.** No hay pruebas unitarias, de integración ni end-to-end incluidas en el repositorio.
8. **Compatibilidad CSS.** `color-mix` requiere navegadores recientes; no se define alternativa visual explícita para versiones antiguas.

12 Mantenimiento y extensión

12.1 Puntos de cambio habituales

- Para añadir un método, extender `cfg.metodo`, `construirMemoria()`, `cargar()`, métricas, controles y textos.
- Para añadir una política, extender los botones, `ALGO_POR_QUE` y `elegir()`.
- Para modificar segmentos, cambiar `SEGS`, el formulario, el modelo y el detalle; hoy varias partes asumen exactamente cuatro.
- Para persistencia, serializar configuración y programas; no conviene persistir IDs de bloques dinámicos porque se regeneran.
- Para pruebas unitarias, extraer la lógica de dominio de la IIFE a un módulo sin DOM y hacer explícitas sus entradas y salidas.

12.2 Pruebas recomendadas

1. partición exacta, proceso un byte/KiB menor y proceso mayor;
2. igualdad de candidatos en mejor y peor ajuste;
3. liberación entre dos huecos y fusión triple;
4. fallo por total insuficiente frente a fallo por falta de continuidad;
5. compactación automática y manual, incluida memoria completamente llena;
6. cambios 1/2/4 MiB de S.O. con todas las tablas variables;
7. nombres con caracteres especiales y payloads HTML;
8. navegación completa por teclado, lector de pantalla y ancho móvil.

13 Conclusión técnica

MemLab implementa de forma compacta y legible un simulador didáctico de asignación contigua. La estructura basada en estado privado, reglas sincrónicas y renderizado centralizado es apropiada para su escala. Los algoritmos principales están representados de manera fiel a su intención educativa: particiones estáticas, división y fusión de huecos, tres ajustes y compactación. Las prioridades de una evolución serían corregir el escape de la bitácora, aclarar casos límite de mensajes y métricas, separar el dominio del DOM e incorporar pruebas automatizadas.